

LoomSet

A 12 Week Build Roadmap

Zero-cost, open-source text data extraction pipeline for ML-ready datasets

Discovery → Scraping → Extraction → Storage → Cleaning → Labeling → Review → Export

Soham Gurav · August 15, 2026

Contents

- Contents2
- How to Use This Roadmap.....3
 - Monthly Phases at a Glance3
- Master Tech Stack4
- Master Concepts List.....5
- Full Reading List (22 Papers)6
- Week-by-Week Plan8
 - Month 18
 - Month 29
 - Month 311
- Final Notes.....14
 - Keep the literature survey close14
 - Zero-cost constraint14
 - If you fall behind14
 - Suggested next step14

How to Use This Roadmap

This roadmap breaks the LoomSet build into 12 weeks (three months), grouped into three monthly phases that follow the pipeline's own architecture: Month 1 builds the acquisition stages (discovery + scraping), Month 2 builds the processing stages (extraction, storage, cleaning), and Month 3 builds the intelligence and delivery stages (labeling, review, export) plus release hardening.

Every week lists five things: the theme, the tech stack to learn, the concepts to study, the specific papers from my literature survey to re-read, and the concrete piece of the pipeline to build. Papers are sequenced so I read each one just before I need it, not all 22 at once.

This assumes roughly 10 to 15 hours/week. If I have more or less time, I can compress or stretch weeks rather than skipping stages as each week's deliverable is a dependency for the next.

Monthly Phases at a Glance

Phase	Focus
Month 1 : Acquisition	Weeks 1 to 4: Project setup, URL discovery, static scraping (Scrapy), dynamic scraping (Playwright).
Month 2 : Processing	Weeks 5 to 8: Content extraction (trafilatura), storage (SQLite/Parquet), classical + LLM-assisted cleaning, labeling theory.
Month 3 : Intelligence & Delivery	Weeks 9 to 12: Auto labeling (Ollama/transformers), human review (Label Studio), export (HF datasets), testing & release.

Mastering the Tech Stack

Everything I'll need to learn across the three months, organized by pipeline stage. I don't need to learn all of this up front, each item is introduced in the week that needs it (see the weekly plan).

Area	Tools / Libraries
Language & Tooling	Python 3.11+, venv/poetry, Git/GitHub, pytest, GitHub Actions
URL Discovery	duckduckgo-search (Python package), SearxNG (self-hosted via Docker)
Static Scraping	Scrapy (spiders, pipelines, middlewares, AutoThrottle)
Dynamic Scraping	Playwright for Python (headless Chromium), optionally scrapy-playwright
Content Extraction	trafilatura
Storage	SQLite (pipeline state/querying), Parquet via pandas/pyarrow (export-ready storage), DuckDB (analytical queries over Parquet)
Auto-Labeling	Ollama (local LLM serving), HuggingFace transformers (zero-shot/text classification)
Human Review	Label Studio (self-hosted)
Export	HuggingFace datasets
Infra	Docker / Docker Compose (for SearxNG and Label Studio)

Master Concepts List

The conceptual foundations underpinning each part of the pipeline, grouped by theme rather than week, so I can use this as a standing checklist.

Theme	Concepts
Scraping & Web	robots.txt/ToS compliance, HTML parsing, CSS/XPath selectors, headless browser rendering, rate limiting, template vs. adaptive extraction
Data Engineering	columnar vs. row storage, data profiling, deduplication/normalization, provenance & lineage, out of memory processing
ML & Labeling	weak supervision & labeling functions, generative denoising of noisy labels, zero shot classification, confidence based human in the loop routing
Systems & Practice	staged/modular pipeline architecture, reproducibility, benchmarking methodology, open source release hygiene

Full Reading List (22 Papers)

Every paper from your literature survey, in the order I'll read them across the 12 weeks. "Week" indicates the primary week each paper is assigned to, some are revisited later when directly relevant.

Week	Paper	Author(s)	Year	Category
Wk1	DEiXTo: A Web Data Extraction Suite	Kokkoras, Ntonas & Bassiliades	2013	Web Scraping
Wk2	RCrawler: An R Package for Parallel Web Crawling and Scraping	Khalil & Fakir	2017	Web Scraping
Wk3	Web Scraping: Applications and Scraping Tools	Matta et al.	2020	Web Scraping
Wk5	CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data	Wenzek et al.	2020	Web Scraping
Wk5	A Framework for Automated Scraping of Structured Data Records (Semantic Scraper)	Kumaresan & Ramanujam	2022	Web Scraping
Wk5	Web Scraping for Data Analytics: A BeautifulSoup Implementation	(IEEE)	2023	Web Scraping
Wk1	Web Scraping: Huge Data Collection from Web	Bhatt et al.	2023	Web Scraping
Wk4	Software Architecture for Improving Scraping Systems Using AI	Posedaru et al.	2024	Web Scraping
Wk4	Scalable and Adaptive Web Scraping Framework for Diverse Data	(ICPCSN)	2024	Web Scraping
Wk1	Automating Web Data Collection: Challenges, Solutions, Python Strategies	Wahed & Bader	2024	Web Scraping
Wk10	LLMs Applied to Web Scraping and Web Crawling: A Systematic Review	(Computing)	2026	Web Scraping
Wk6	A Model and System for Querying Provenance from Data Cleaning Workflows	(ADBIS)	2021	Data Cleaning
Wk7	Auto-Prep: Efficient and Automated Data Preprocessing Pipeline	Bilal et al.	2022	Data Cleaning
Wk6	Data Cleaning and Machine Learning: A Systematic Literature Review	Côté et al.	2023	Data Cleaning
Wk7	IterClean: An Iterative Data Cleaning Framework with LLMs	Ni et al.	2024	Data Cleaning
Wk6	Trends and Challenges in Data Cleaning for Large-Scale Systems	(IJGIS)	2025	Data Cleaning
Wk8	Data Programming: Creating Large Training Sets, Quickly	Ratner et al.	2016	Labeling
Wk8	Snorkel: Rapid Training Data Creation with Weak Supervision	Ratner et al.	2017	Labeling

Week	Paper	Author(s)	Year	Category
Wk8	Snuba: Automating Weak Supervision to Label Training Data	Varma & Ré	2018	Labeling
Wk9	Semi-Automated Data Labeling	Desmond et al.	2021	Labeling
Wk9	Auto-labelling Entities in Low-Resource Text: A Geological Case Study	Enkhsaikhan et al.	2021	Labeling
Wk10	Webscraping Data Labeling System on LC-MS of Rodent Tuber	(2022)	2022	Labeling
Wk9	MADLaP: Multistep Automated Data Labelling Procedure (Thyroid Nodules)	Zhang et al.	2023	Labeling

Source: *LoomSetLiterature_Survey.pdf* : A Literature Survey for LoomSet, Soham Gurav, August 2026.

Week by Week Plan

Month 1

Week 1: Project Groundwork & Scraping Ethics

Theme	Set up the repo, environment, and re ground the plan in my existing docs before writing pipeline code.
Tech Stack	– Python 3.11+ virtual environments (venv/poetry) – Git/GitHub project hygiene (issues, branches, README) – Docker basics (I'll need it for SearxNG later)
Concepts	– robots.txt and Terms of Service compliance for scraping – Legal/ethical boundaries of web scraping at scale – Pipeline architecture patterns (staged/modular ETL)
Papers	– Wahed & Bader (2024) - Automating Web Data Collection: Challenges, Solutions, and Python-Based Strategies – Bhatt et al. (2023) - Web Scraping: Huge Data Collection from Web
Deliverable	Repo scaffolded with the 8 stage architecture as empty modules; re read research_questions.md, architecture.md, and limitations.md and note any changes needed before coding begins.
Milestone	Environment ready, ethics/compliance stage designed on paper.

Week 2: URL Discovery

Theme	Build the first working stage: turning a niche topic into a seed list of candidate URLs.
Tech Stack	– Duckduckgo search (Python package) – SearxNG self-hosting via Docker Compose – Query construction & pagination for search APIs
Concepts	– Rate limiting and politeness windows – Query formulation for low resource/niche topics – Deduplicating and scoring discovered URLs by relevance
Papers	– Khalil & Fakir (2017) - RCrawler: An R Package for Parallel Web Crawling and Scraping (motivates why discovery must be automated, not manual)
Deliverable	`url_discovery` module: given a topic string, returns a deduplicated, relevance-scored list of seed URLs from duckduckgo-search and/or a self-hosted SearxNG instance.
Milestone	Stage 1 (URL Discovery) working end to end on a test niche topic.

Week 3: Static Scraping with Scrapy

Theme	Learn Scrapy deeply and build the static page scraping stage.
Tech Stack	– Scrapy: spiders, item pipelines, middlewares, settings – CSS selectors & XPath – Scrapy's concurrency/throttling controls (AutoThrottle, CONCURRENT_REQUESTS)
Concepts	– HTML parsing fundamentals – Template based vs. adaptive extraction rules – robots.txt enforcement inside a crawler (not just in theory)
Papers	– Kokkoras, Ntonas & Bassiliades (2013) - DEiXTo: A Web Data Extraction Suite – Matta et al. (2020) - Web Scraping: Applications and Scraping Tools
Deliverable	`static_scraper` Scrapy project that crawls the seed URLs from Week 2, respects robots.txt, and stores raw HTML with source metadata.
Milestone	Stage 2 (Static Scraping) working with AutoThrottle and robots.txt enforcement.

Week 4: Dynamic Scraping with Playwright

Theme	Handle JavaScript rendered pages that Scrapy alone can't reach, and merge this into the scraping stage.
Tech Stack	– Playwright for Python: browser contexts, page.goto, wait strategies – Headless Chromium automation – scrapy-playwright integration (or a standalone Playwright fallback path)
Concepts	– DOM-after-render vs. raw HTML – Detecting when a page needs a headless browser vs. a simple GET – Basic anti-bot/consent-banner handling
Papers	– Scalable and Adaptive Web Scraping Framework for Extracting Diverse Data from Open Internet Sources (2024) – Posedaru et al. (2024) - Software Architecture for Improving Scraping Systems Using Artificial Intelligence (note: skip their paid-ChatGPT selector step - you'll do this with Ollama in Month 3 instead)
Deliverable	`dynamic_scraper` module using Playwright; a simple router decides static vs. dynamic path per URL. Both scrapers feed a unified raw-page store.
Milestone	MONTH 1 MILESTONE: URL discovery -> static scraping -> dynamic scraping fully wired together and runnable as one command on a real niche topic.

Month 2

Week 5: Content Extraction & Storage Layer

Theme	Turn raw HTML into clean text and land it in a queryable, ML-friendly store.
Tech Stack	- trafilatura API (extract, bare_extract, metadata options)

	– SQLite (schema design, indexing) – Parquet via pandas/pyarrow
Concepts	– Boilerplate/nav removal in content extraction – Columnar vs. row storage trade-offs (why Parquet for export, SQLite for querying) – Niche-topic relevance filtering before storage
Papers	– Kumaresan & Ramanujam (2022) - Semantic Scraper (why generic template-based extraction breaks down) – Web Scraping for Data Analytics: A BeautifulSoup Implementation (2023) (why hand-tuned selectors don't generalize - motivates trafilatura) – Wenzek et al. (2020) - CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data (for the relevance-filter idea)
Deliverable	`extraction` stage (trafilatura) + `storage` stage (SQLite for pipeline state, Parquet for extracted content) with a topic-relevance filter before writes.
Milestone	Stage 4 + 5 (Extraction + Storage) complete; can run discovery-to-storage on a real topic and inspect clean text in SQLite/Parquet.

Week 6: Data Cleaning Foundations

Theme	Learn classic data cleaning and add a provenance-aware cleaning stage.
Tech Stack	– pandas for tabular cleaning – DuckDB for out-of-memory/SQL-style cleaning over Parquet – Basic data profiling tools
Concepts	– Data quality dimensions (completeness, consistency, dedup, validity) – Deduplication and normalization strategies for scraped text – Provenance/lineage tracking through a pipeline
Papers	– Côté et al. (2023) - Data Cleaning and Machine Learning: A Systematic Literature Review – Trends and Challenges in Data Cleaning for Large-Scale Systems (2025) – A Model and System for Querying Provenance from Data Cleaning Workflows (2021)
Deliverable	`cleaning` module: dedup, normalization, and a provenance log written alongside the SQLite/Parquet store, applied to Week 5's extracted data.
Milestone	Baseline cleaning stage running on real extracted data, with a lineage record per row.

Week 7: LLM-Assisted Cleaning with Ollama

Theme	Add a local-LLM cleaning pass for the errors classic rules can't catch.
Tech Stack	– Ollama installation & local model serving (e.g. Llama 3 / Mistral class models) – Prompt design for structured cleaning tasks – Basic early-stopping/iteration-cap logic

Concepts	– LLM-in-the-loop error detection and repair – Cost/latency trade-offs of iterative LLM prompting (and why local models remove the monetary cost) – When to trust a rule vs. an LLM judgment
Papers	– Ni et al. (2024) - IterClean: An Iterative Data Cleaning Framework with Large Language Models – Bilal et al. (2022) - Auto-Prep: Efficient and Automated Data Preprocessing Pipeline
Deliverable	Ollama-backed cleaning pass added to `cleaning`, capped by an early-stopping rule, with a manual override path to skip it for known-good data types.
Milestone	MONTH 2 MID-POINT: full pipeline runs discovery -> scraping -> extraction -> storage -> cleaning end-to-end.

Week 8: Weak Supervision & Labeling Theory

Theme	Study the theory behind auto-labeling before writing the labeling stage - this week is reading-and-design heavy.
Tech Stack	– HuggingFace `transformers` pipelines (zero-shot-classification, text-classification) – Reading Snorkel's/Snuba's public docs or code for design ideas (no need to install Snorkel itself)
Concepts	– Weak supervision & labeling functions – Generative denoising models for combining noisy labels – Bootstrapping a seed set for low-resource domains
Papers	– Ratner et al. (2016) - Data Programming: Creating Large Training Sets, Quickly – Ratner et al. (2017) - Snorkel: Rapid Training Data Creation with Weak Supervision – Varma & Ré (2018) - Snuba: Automating Weak Supervision to Label Training Data
Deliverable	Design doc: a small library of reusable labeling-function templates per niche domain, plus a baseline zero-shot classifier via `transformers` for comparison.
Milestone	Labeling strategy designed and validated against the literature before implementation.

Month 3

Week 9: Auto-Labeling with Local LLMs

Theme	Implement the auto-labeling stage using Ollama, with confidence scoring for what to escalate to a human.
Tech Stack	– Ollama for labeling prompts (structured/JSON output) – Confidence scoring & thresholding logic – HuggingFace `transformers` as a secondary/ensemble labeler
Concepts	– Human-in-the-loop routing (only escalate low-confidence items) – Domain jargon / low-resource transfer limits of general models

	– Combining multiple weak signals into one label
Papers	– Desmond et al. (2021) - Semi-Automated Data Labeling – Enkhsaikhan et al. (2021) - Auto-labelling Entities in Low-Resource Text: A Geological Case Study – Zhang et al. (2023) - MADLaP: Multistep Automated Data Labelling Procedure
Deliverable	`auto_labeling` stage: Ollama + transformers produce labels with confidence scores on the cleaned dataset; low-confidence items flagged for review.
Milestone	Stage 6 (Auto-Labeling) producing a labeled dataset with a measurable low-confidence fraction.

Week 10: Human Review with Label Studio

Theme	Close the loop: route flagged items to a real annotation UI and merge reviewed labels back in.
Tech Stack	– Label Studio setup (local, Docker or pip) – Label Studio project/config (labeling interface XML) – Import/export via Label Studio's API
Concepts	– Annotation UX for domain experts – Merging human-reviewed labels back into the training set – Basic inter-annotator agreement awareness
Papers	– LLMs Applied to Web Scraping and Web Crawling: A Systematic Review (2026) - revisit for the hybrid human+model pattern – Webscraping Data Labeling System on LC-MS of Rodent Tuber (2022) - lesson on caching unreliable reference sources locally
Deliverable	`human_review` stage: low-confidence items from Week 9 flow into Label Studio; reviewed labels merge back into the dataset.
Milestone	Stage 7 (Human Review) integrated; full auto-label + review loop functioning on real data.

Week 11: Export & End-to-End Integration

Theme	Wire every stage into one runnable pipeline and produce a real ML-ready dataset.
Tech Stack	– HuggingFace `datasets` library (Dataset/DatasetDict, push_to_hub optional) – Dataset card / metadata conventions – CLI or config-driven pipeline orchestration (e.g. a single `run_pipeline.py` with stage toggles)
Concepts	– ML-ready dataset formatting standards – Train/validation/test splitting for niche datasets – Reproducibility (pinned versions, seeds, run configs)

Papers	– Revisit CCNet (2020) and Côté et al. (2023) for export/benchmarking conventions worth adopting
Deliverable	End-to-end run on a real niche topic, discovery through export, producing a versioned HuggingFace `datasets`-compatible dataset with a dataset card.
Milestone	MONTH 3 MID-POINT: full 8-stage pipeline runs unattended on a real topic end-to-end.

Week 12: Testing, Documentation & Release

Theme	Harden, document, and ship v0.1 as a genuinely usable open-source project.
Tech Stack	– pytest for unit/integration tests – GitHub Actions CI – Packaging (pyproject.toml, pip-installable structure)
Concepts	– Reproducibility & open-source release hygiene – Benchmarking your own pipeline (throughput, extraction precision) the way the surveyed papers did – Writing a limitations section honestly (you already have a draft - stress-test it against what you built)
Papers	– Skim your own literature survey's summary table end-to-end as a checklist: for every limitation you flagged, confirm the pipeline actually implements the mitigation you proposed.
Deliverable	v0.1 tagged release: README, updated architecture.md/limitations.md reflecting what was actually built, CI passing, and a short benchmark note (throughput + labeling accuracy on a sample).
Milestone	MONTH 3 MILESTONE: LoomSet v0.1 released publicly with docs, tests, and a working example dataset.

Final Notes

Keep the literature survey close

Every stage in this roadmap maps directly to a limitation identified in your literature survey and the mitigation you already proposed in its summary table. Week 12 asks you to re-check that mapping against what you actually built — do this honestly; it's the strongest part of the project's academic framing.

Zero-cost constraint

Every tool in the Master Tech Stack is free and open-source. The only "cost" is compute and time — Ollama models run locally, SearxNG and Label Studio are self-hosted, and HuggingFace datasets/transformers are free to use without paid APIs.

If you fall behind

- Protect Weeks 4, 8, and 12 — they're phase-closing milestones the following month depends on.
- It's fine to compress Weeks 1 and 6 (foundation-heavy, lighter on new code) if you need to recover time.
- Cut scope before cutting stages — a pipeline that runs end-to-end on one niche topic beats a pipeline with one perfect stage and four missing ones.

Suggested next step

Once this roadmap is underway, consider a gap analysis cross-referencing the literature survey against newer papers (2026+) before Week 12's write-up, so the final limitations.md reflects the most current state of the field.